

SiSH — Simple Shell

1. Overview

SiSH is a minimal POSIX-style shell that:

- reads a command line from the user,
- resolves the executable via the PATH environment variable (or absolute/relative path),
- creates a child process (fork), replaces the child image with the requested program (execve),
- waits for completion (waitpid) before prompting again.

This assignment demonstrates UNIX process control and environment handling (PATH, envp).

2. Features

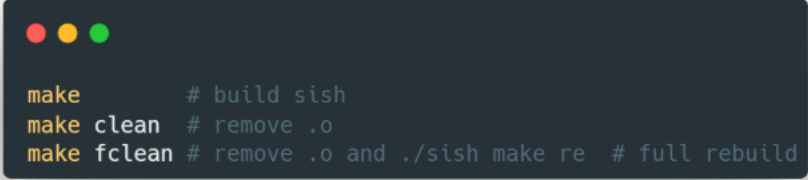
- PATH lookup: Scans each directory in \$PATH for the command name; absolute (/...) and relative (./...) paths run directly.
- Arguments: Tokenizes the input into argv[] and passes all args to execve.
- Blocking execution: Parent waits for the child to finish, then re-prompts.
- Small helpers: my_strncat safely extracts substrings.

3. Project Layout

- shell.c — shell loop, PATH resolution, forking/execution, argument parsing
- strncat.c — my_strncat helper
- Makefile — build script

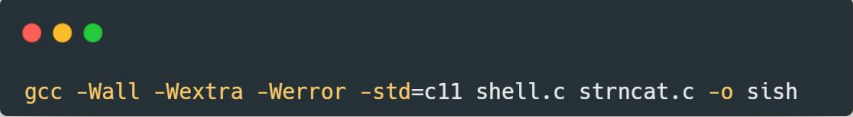
4. Build Instruction

Using Make Command:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains three lines of text: 'make # build sish', 'make clean # remove .o', and 'make fclean # remove .o and ./sish make re # full rebuild'.

```
make # build sish
make clean # remove .o
make fclean # remove .o and ./sish make re # full rebuild
```

Direct Compile:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a single line of text: 'gcc -Wall -Wextra -Werror -std=c11 shell.c strncat.c -o sish'.

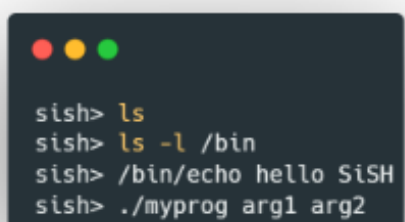
```
gcc -Wall -Wextra -Werror -std=c11 shell.c strncat.c -o sish
```

5. Usage

Start the shell with `./sish`. The prompt appears as `"sish>"`. Enter any executable (with or without arguments).

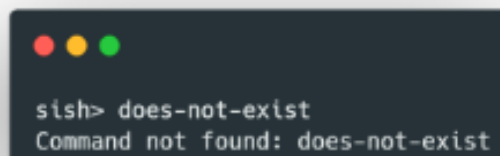
SiSH forks, the child executes the program with `execve`, the parent waits (`waitpid`), then the prompt reappears.

Examples:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays four lines of text: a prompt followed by 'ls', a prompt followed by 'ls -l /bin', a prompt followed by '/bin/echo hello SiSH', and a prompt followed by './myprog arg1 arg2'.

```
sish> ls
sish> ls -l /bin
sish> /bin/echo hello SiSH
sish> ./myprog arg1 arg2
```

Error Example:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It displays two lines of text: a prompt followed by 'does-not-exist', and the next line showing the error message 'Command not found: does-not-exist'.

```
sish> does-not-exist
Command not found: does-not-exist
```

6. Implementation Notes

- PATH resolution: If the input doesn't start with / or ./, split \$PATH by ':' and check `access(dir/command, X_OK)`.
- Argument parsing: Split on spaces and build a NULL-terminated `argv` (simple tokenizer; no quoting/escaping yet).
- Execution: Parent calls `fork()`; child runs `execve(resolved_path, argv, envp)`; parent calls `waitpid`.
- Memory hygiene: Free allocated buffers and paths on every branch.